

Product Requirements Document

Fake Offer Letter & Phishing Inspector

Hackathon MVP • 3-hour execution plan • Bento-first product

1. Product Summary

A single-page security scanner that helps job seekers inspect suspicious job offers, appointment letters, and related URLs. The product parses submitted text or a URL, checks scam indicators such as payment demands and domain information, and returns a dynamic Scam Threat Index from 0–100% with evidence-based explanations.

2. Problem

Job seekers and renters can encounter fake appointment letters, pay-for-equipment phishing, and deposit traps that may bypass conventional spam filters. The product should turn a potentially confusing message into an understandable risk report.

3. Goals

- Provide a fast, single-page scan experience.
- Accept either job-offer text or a URL.
- Identify payment-demand and other suspicious signals.
- Check domain-age information when a URL is provided.
- Produce a 0–100% Scam Threat Index.
- Show the evidence behind the score instead of presenting a black-box verdict.
- Store scan results remotely using Firebase Firestore.
- Be polished enough for a live hackathon demonstration.

4. Non-Goals for the MVP

- No browser extension.
- No full website crawler or sandbox.
- No autonomous reporting to police, employers, banks, or platforms.
- No complex user profile system.
- No admin dashboard.
- No custom machine-learning model training.
- No attempt to guarantee that an offer is genuinely safe or fraudulent.

5. Target Users

User	Need	Primary Action
Job seeker	Quickly understand whether an offer contains scam indicators.	Paste offer text / URL and scan.
Student / fresher	Check an unfamiliar recruiter before paying or sharing information.	Review risk score and evidence.
Hackathon judge	Understand the value immediately during a demo.	Try a prepared suspicious offer and inspect the report.

6. Core User Journey

1. User opens the landing/scanner page.
2. User selects Text or URL input mode.
3. User pastes the suspicious offer or URL.
4. User clicks Analyze.
5. Frontend sends the request to the Flask API.
6. Backend extracts signals and calls the AI analysis service.
7. If a URL is supplied, backend obtains domain-age information from the configured domain service.
8. Backend combines signals into a transparent 0–100% threat score.
9. Frontend displays risk level, score, detected red flags, evidence, and AI explanation.
10. The scan result is stored in Firebase Firestore without requiring a local database.

7. Functional Requirements

ID	Requirement	Priority
FR-01	Accept job-offer / appointment-letter text.	Must
FR-02	Accept a URL related to the offer.	Must
FR-03	Analyze payment-demand red flags.	Must
FR-04	Analyze urgency / pressure language.	Must
FR-05	Analyze suspicious contact / communication patterns.	Should
FR-06	Check domain age for URL input.	Must
FR-07	Generate Scam Threat Index from 0–100%.	Must
FR-08	Display evidence for each detected signal.	Must
FR-09	Generate a concise AI	Must

	explanation.	
FR-10	Persist scan result in Firebase Firestore.	Should
FR-11	Provide demo/example inputs.	Should
FR-12	Handle invalid/empty input gracefully.	Must

8. Threat Scoring Model

The AI should identify and structure evidence; the application should calculate the final score using deterministic weights. This makes the result explainable and avoids letting a language model silently invent the final percentage.

Signal	Illustrative Weight	Example
Upfront payment / equipment fee	+30	Pay a fee before joining.
Deposit / refundable security demand	+25	Send a deposit to confirm the role.
Strong urgency / pressure	+15	Offer expires in two hours.
Suspicious communication channel	+10	Move only to Telegram/WhatsApp.
Sensitive information request	+10	Unusual request for identity/bank details.
Other validated red flag	+10	Pattern identified by analysis.

The weights are an implementation proposal for the MVP, not a claim that a specific percentage proves fraud. The UI should describe the result as a threat/risk assessment based on detected signals.

9. AI Output Contract

The AI service should return structured JSON similar to:

```
{
  "flags": [
    {"title": "Payment request", "severity": "high", "evidence": "Pay ₹4,999 for equipment"}
  ],
  "summary": "Multiple signals require caution.",
  "recommended_action": "Do not pay or share sensitive information until independently verified."
}
```

10. Data & Privacy

- Use Firebase Firestore as the cloud database; no local database is required in production.

- Store scan metadata and result fields needed for the demo.
- Avoid storing unnecessary personal information.
- Do not store passwords, payment credentials, or sensitive identity documents.
- Clearly tell users that the scanner provides an assessment, not a definitive legal/fraud determination.

11. Technical Stack

Layer	Technology	Purpose
Frontend	HTML5 + CSS3 + Vanilla JavaScript	Single-page interface and interactions.
Backend	Python + Flask	API, scoring, orchestration, validation.
AI	LLM API	Extract suspicious signals and produce explanation.
Domain intelligence	Domain / WHOIS API	Domain-age signal for URL scans.
Cloud database	Firebase Firestore	Persist scan results remotely.
Deployment	Render	Host Flask application.
Source control	Git + GitHub	Versioning and deployment workflow.

12. API Surface

Endpoint	Method	Purpose
/api/analyze	POST	Analyze text or URL and return a threat report.
/api/health	GET	Simple service health check.

13. Acceptance Criteria

- A user can submit a realistic suspicious job offer and receive a result without refreshing the page.
- The result contains a 0–100% Scam Threat Index and a risk label.
- At least three concrete red flags can be shown for a prepared suspicious example.
- Each displayed red flag includes supporting evidence.
- URL scans include domain-age information when the external service responds successfully.
- The application still returns a useful text-only result if domain lookup fails.
- A scan result can be written to Firestore.
- The interface is responsive and usable on a laptop/mobile-sized viewport.
- No critical API keys are exposed in frontend JavaScript.

14. 3-Hour Hackathon Execution Plan

Time	Work
0:00–0:20	Project setup, Flask server, environment variables, Firebase connection.
0:20–0:50	Build bento landing/input UI.
0:50–1:20	Implement /api/analyze and deterministic scoring.
1:20–1:45	Integrate AI analysis and domain-age lookup.
1:45–2:05	Connect Firestore and save scan results.
2:05–2:15	Error states and end-to-end test.
2:15–3:00	Polish UI, responsive behavior, demo data, deployment, final rehearsal.

15. Demo Scenario

Prepare one strong suspicious offer, one medium-risk example, and one relatively clean example. During the demo: paste the suspicious offer → scan → show the score → reveal the evidence → show the domain-age signal → explain that the score is assembled from observable indicators.